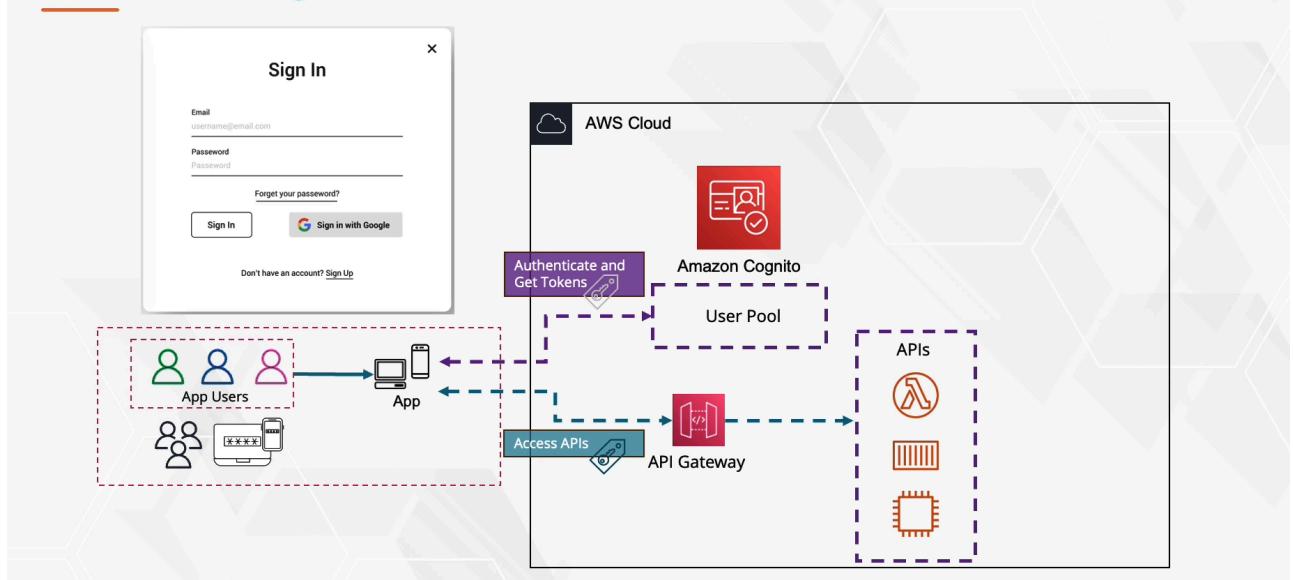


Amazon Cognito

- Identity service that is designed for your application users.
1. User Pools - Allow you to define user, and user groups to separate out different types of users. You can also design user interface to allow users to signup and signin.

Amazon Cognito –Application Users – User Pools



2. Identity Pools - create identities - assign roles based on the various identity pools

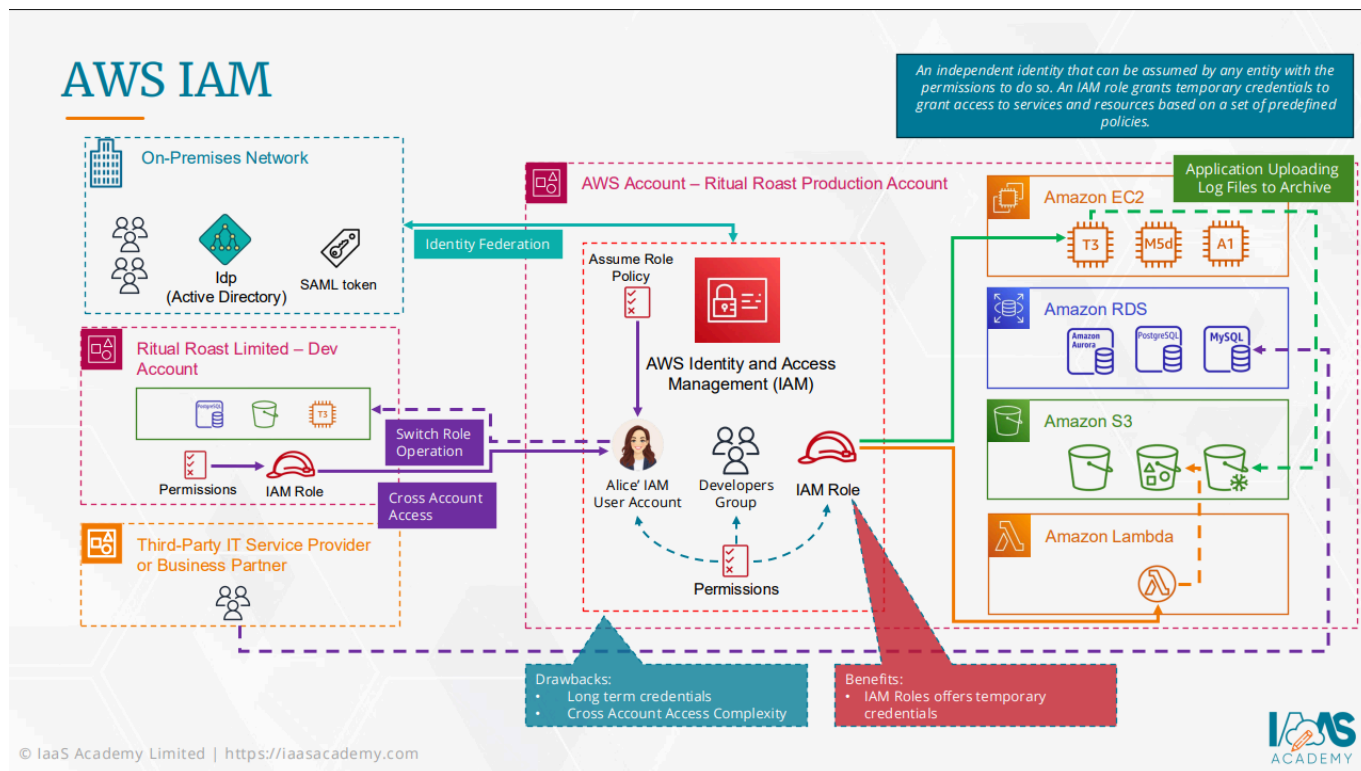
AWS IAM

IAM user Account + IAM Policies (policies)

IAM Groups - Developers Groups, Admin Groups (Assign users to the required group)

IAM Role -- An **independent identity that can be assumed by any entity with the permissions to do so**. An IAM role grants temporary credentials to grant access to services and resources based on a set of predefined policies.

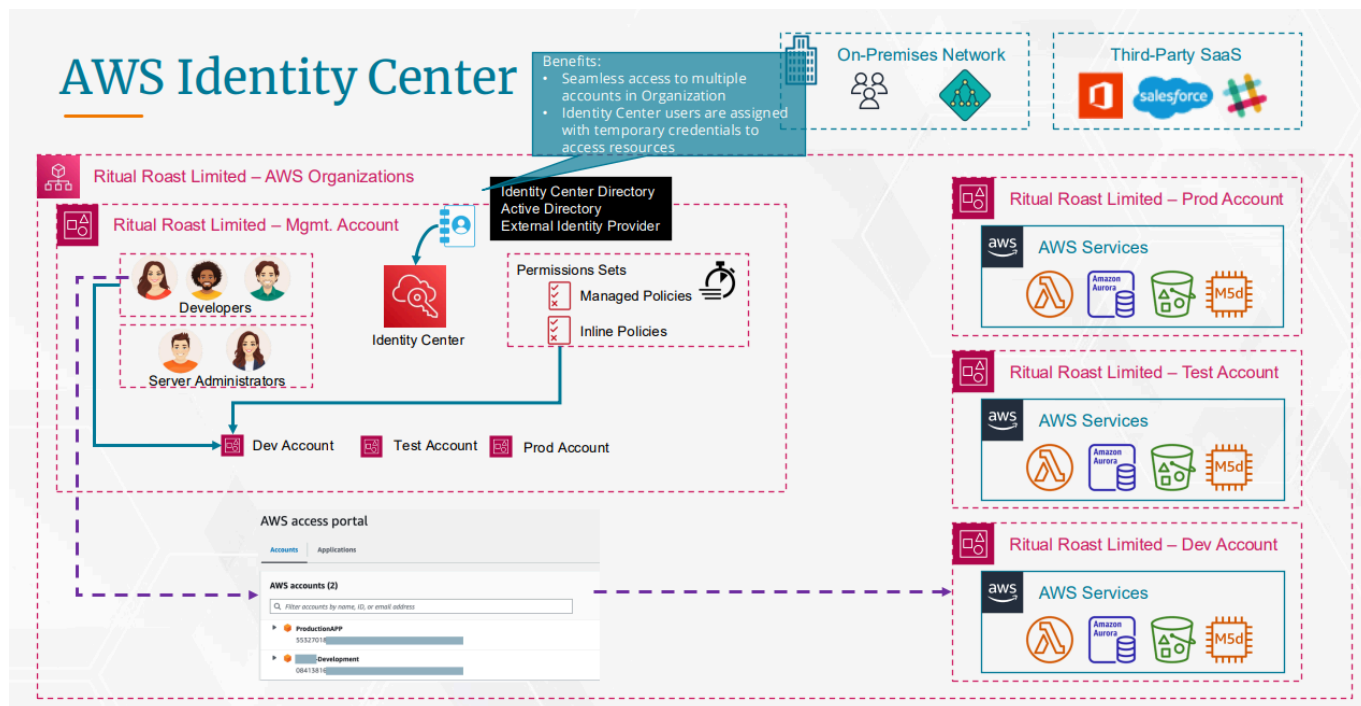
- The original Identity service provided by amazon
- You can create a IAM role and add policies to it, so if a user(rohit) needs access to create the resources within the scope of that IAM role. For that IAM role you can define a **trust policy** - identifying who can be tested to assume that role in our case it is user(rohit) using account id and user(rohit). So now the user can perform **Role switch Operation** to assume that IAM role and create resources in that other account for say. The user is created in production account now you want to create resources in development account, you don't need to create the same user in development account as well that becomes two accounts. You can perform the above operation for **Cross account Access**. Creating seperate account for each user is not the problem, what about if the users are 200 - complexity increases with the scale.



- IAM Roles uses **Amazon STS** - Amazon Security Token Service to generate temporary dynamic credentials to and provides them to the entity that is assuming that role when required. That credentials expire after a short period of time. you don't need to worry about password rotation or key rotation.

AWS Identity Center

- It is the centralized center to manage workforce access to multiple aws accounts and applications. I enables secure SSO by integrating directly with the identity providers like OKTA, Google Workforce.
- The identity provider can be
 - The native Identity Center Directory
 - External identity provider like OKTA, PING
 - Active Directory
- **Permission Sets** - this is what defines that what your users can or cannot do in their accounts.
- With groups and permission sets in place, you can specify which groups can access those accounts and what permissions do they get.



IAM Policies

- AWS Managed Policies - created and maintained by AWS
- Customer Managed Policies - created and managed by users
- Inline Policies - specially for an identity, directly attached.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VisualEditor0",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::ritual-roast-source-code",
        "arn:aws:s3:::ritual-roast-source-code/*"
      ]
    },
    {
      "Sid": "visutaleditor1",
      "Effect": "Allow",
      "Actions": "s3:ListAllMyBuckets",
      "Resource": "*"
    }
  ]
}
```

- Version: 2012-10-17

- Statement: Can have multiple statement blocks.
- Sids: Statement IDs - Not mandatory - can chose as you like
- Effect: Allow or Deny
- Action: List of actions that this policy can allow or denies
- Resource: list of resources to which this policy applies to

ARN Formats

arn:aws:ec2:us-east-1:987641320:instance/i-243iulfl245jk arn:partition:service:region:account-id:resource-id
 arn:partition:service:region:account-id:resource-type/resource-id

Bucket Policy Templates

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllBucketActions",
      "Effect": "Allow",
      "Action": "s3:*Object",
      "Resource": "*"
    }
  ]
}
```

Condition Based deny when the requests come from ip addresses outside of the allowed ranges.

- **IAM policies is by default have deny all principal, before your actually filter anything to deny. You need to allow all so then only you can filter to deny.**

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["ec2:TerminateInstances"],
      "Resource": "*"
    },
    {
      "Effect": "Deny",
      "Action": ["ec2:TerminateInstances"],
      "Condition": {
        "NotIpAddress": {
          "aws:SourceIp": {
            "192.0.2.0/24",
            "203.0.113.0/24"
          }
        }
      }
    }
  ]
}
```

```

    "Resource": ["*"]
  }
]
}

```

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ec2:*",
      "Resource": "*"
    },
    {
      "Effect": "Deny",
      "Action": [
        "ec2:StopInstances",
        "ec2:TerminateInstances"
      ],
      "Resource": "*",
      "Condition": {
        "BoolIfExists": {"aws:MultiFactorAuthPresent": false}
      }
    }
  ]
}

```

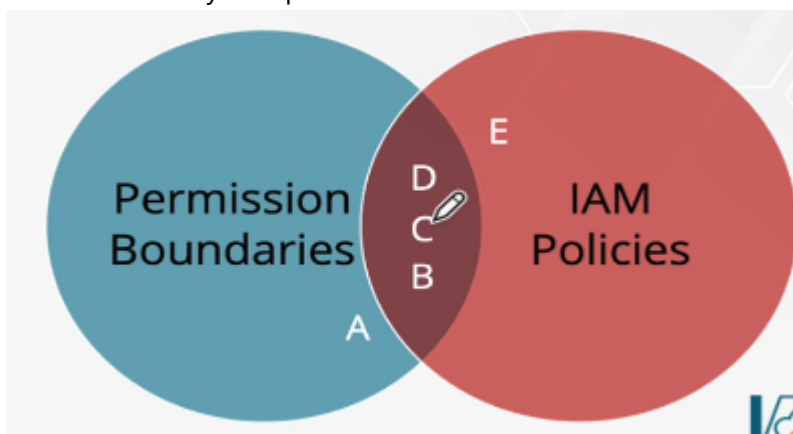
```

{
  "Version": "2012-17-10",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:StopInstance",
        "ec2:TerminateInstance"
      ],
      "Resource": "arn:aws:ec2:*:*:instance:/*",
      "Condition": {
        "StringEquals": {
          "aws:ResourceTag/Owner": "${aws:username}"
        }
      }
    },
    {
      "Effect": "Deny",
      "Action": [
        "ec2:StopInstances",
        "ec2:TerminateInstances"
      ],
      "Resource": "*"
    }
  ]
}

```

IAM Permission Boundaries

- Are another set of boundaries you can assign another user or role.
- Permission boundaries define the maximum permissions that an identity-based policy can grant to an IAM entity. An entity's permissions boundary allows it to perform only the actions that are allowed by both its identity-based policies and its permission boundaries.
- Which basically means both permissions boundaries and IAM policy should be aligned to get access to any service.
- Incase if a user is having s3, cloudwatch and IAM full access and also want to have access to EC2, what if the user having access to IAM creates/gives access to EC2 to himself. In this case this can be stopped by creating a permission boundary for that entity(user/role) so that even if the user grants EC2 access to himself, he would be not able to have EC2 access as his access is not aligned with permission boundary.
- So the access of the identity is depend on the combination of Permission boundary and IAM Policy of



the entity.

- It is not just permission boundaries, even the entity should have correct IAM policies to have access even though they have permission boundaries.

Permission Boundary Policy JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAdminAccess",
      "Effect": "Allow",
      "Action": "*",
      "Resource": "*"
    },
    {
      "Sid": "DenyAccessToCostAndBilling",
      "Effect": "Deny",
      "Action": [
        "account:*",
        "aws-portal:*",
        "savingsplans:*",
        "cur:*",
        "ce:*"
      ],
      "Resource": "*"
    }
  ]
}
```

```

{
  "Sid": "DenyPermBoundaryIAMPolicyAlteration",
  "Effect": "Deny",
  "Action": [
    "iam:DeletePolicy",
    "iam:DeletePolicyVersion",
    "iam:CreatePolicyVersion",
    "iam:SetDefaultPolicyVersion"
  ],
  "Resource": [
    "arn:aws:iam::YourAccount_ID:policy/ScopePermissions"
  ]
},
{
  "Sid": "DenyRemovalOfPermBoundaryFromAnyUserOrRole",
  "Effect": "Deny",
  "Action": [
    "iam:DeleteUserPermissionsBoundary",
    "iam:DeleteRolePermissionsBoundary"
  ],
  "Resource": [
    "arn:aws:iam::YourAccount_ID:user/*",
    "arn:aws:iam::YourAccount_ID:role/*"
  ],
  "Condition": {
    "StringEquals": {
      "iam:PermissionsBoundary":
"arn:aws:iam::YourAccount_ID:policy/ScopePermissions"
    }
  }
},
{
  "Sid": "DenyAccessIfRequiredPermBoundaryIsNotBeingApplied",
  "Effect": "Deny",
  "Action": [
    "iam:PutUserPermissionsBoundary",
    "iam:PutRolePermissionsBoundary"
  ],
  "Resource": [
    "arn:aws:iam::YourAccount_ID:user/*",
    "arn:aws:iam::YourAccount_ID:role/*"
  ],
  "Condition": {
    "StringNotEquals": {
      "iam:PermissionsBoundary":
"arn:aws:iam::YourAccount_ID:policy/ScopePermissions"
    }
  }
},
{
  "Sid": "DenyUserAndRoleCreationWithOutPermBoundary",
  "Effect": "Deny",
  "Action": [
    "iam:CreateUser",

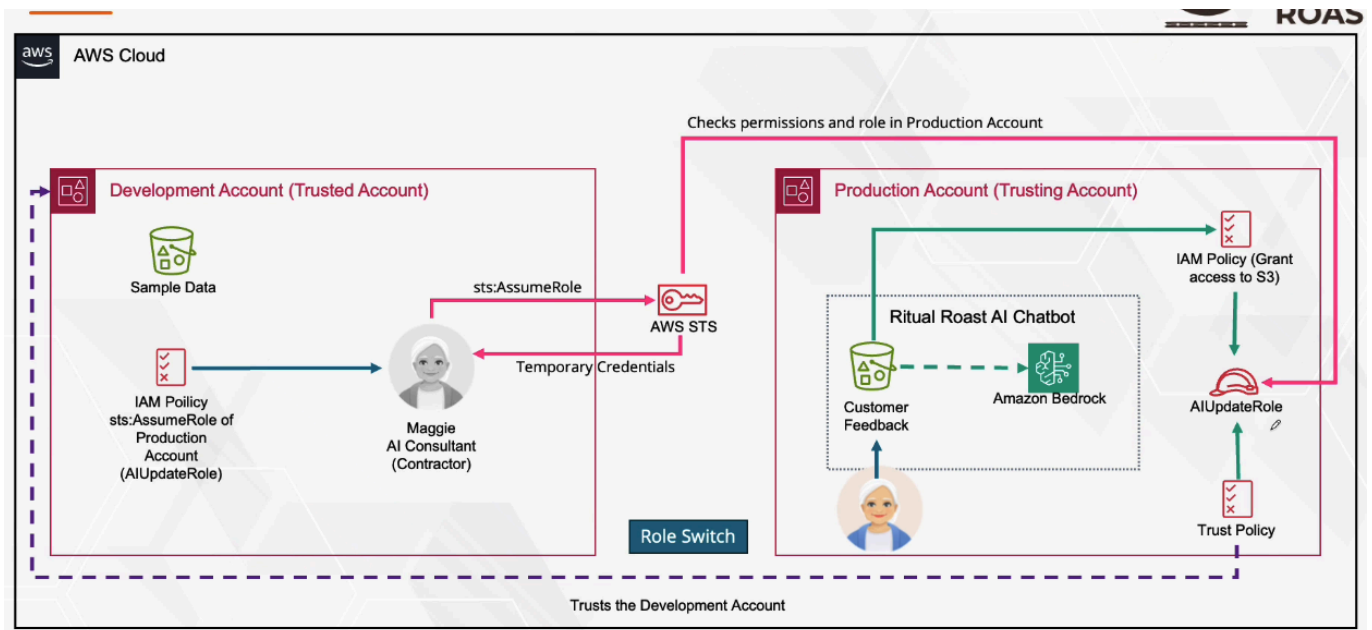
```

```

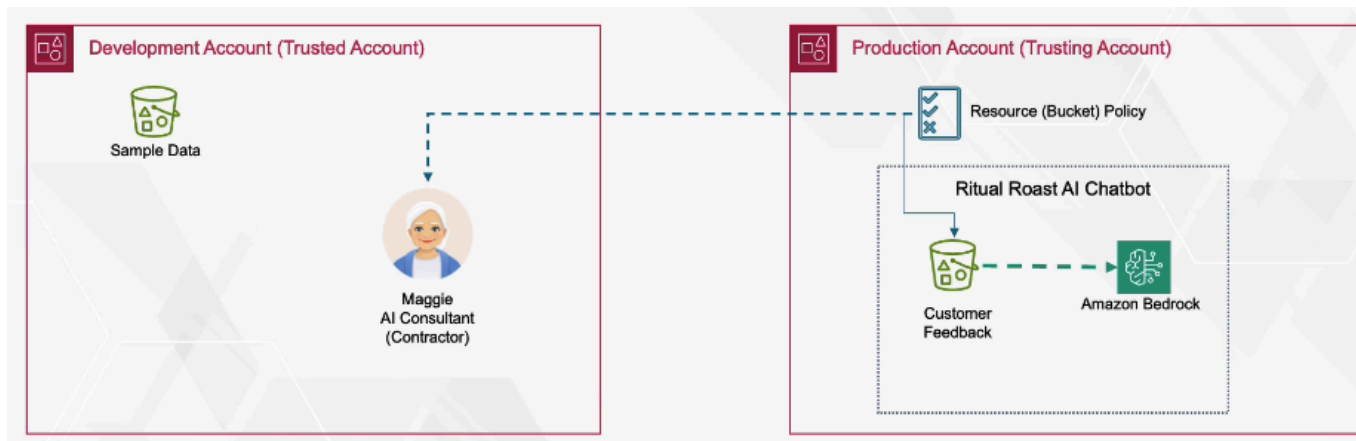
    "iam:CreateRole"
  ],
  "Resource": [
    "arn:aws:iam::YourAccount_ID:user/*",
    "arn:aws:iam::YourAccount_ID:role/*"
  ],
  "Condition": {
    "StringNotEquals": {
      "iam:PermissionsBoundary":
        "arn:aws:iam::YourAccount_ID:policy/ScopePermissions"
    }
  }
},
{
  "Sid": "DenyIAMActions",
  "Effect": "Deny",
  "Action": "iam:PassRole",
  "Resource": "arn:aws:iam::YourAccount_ID:role/*"
}
]
}

```

Use case for cross account access



Resource Based Policy



- For cross account access, you are creating IAM role in the production account, attaching a trust policy to it to trust the dev account, attaching s3 policy to the role in production account. And the user in the dev account should have the IAM policy to assume STS role in the prod account - a lot of hassle.
- Without all of the above, you can just use resource based policy to the resource you want to access.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::747289879815:user/maggie"
      },
      "Action": [
        "s3:GetObject",
        "s3:ListBucket",
        "s3:PutObject"
      ],
      "Resource": [
        "arn:aws:s3:::rohit-backend-terraform",
        "arn:aws:s3:::rohit-backend-terraform/*"
      ]
    }
  ]
}
```

IAM Policy Evaluation

Types of policies

- **IAM Policies** - are attached to an IAM identity
- **Resource based policies** - attached to a specific resource and applied to a principal
- **Permission boundaries** - sets the maximum permission an IAM policy can grant an IAM entity.
- **Service Control Policies (SCP)** - specify maximum permissions for Organization or OU and apply to principals in member accounts, including root users.
- **Session Policies** - allows you to define policies for temporary sessions for a role or federated user, using the AssumeRole* API operations:

- *AssumeRole* to assume a role
- *AssumeRoleWithSAML* for identities authenticated with a SAML 2.0 compatible identity provider.
- *AssumeRoleWithWebIdentity* for authentication with web identity providers, e.g. OAuth 2.0

Identity Center

- Allows you to access all your AWS accounts in a central view.

Directory Services

- A directory services is a database that stores and manages information about users and resources on a network.

